Challenge - MLFlow - Fast API - Asignatura Introducción a MLOps

Lidia Lago Cordero

Mi ordenador utiliza el sistema operativo Ubuntu. Ya tengo Miniconda instalado.
Creamos el entorno de nuestro proyecto en la consola, lo llamaremos 'challenge_mlops':

```
conda create -n challenge_mlops python=3.10 -y
conda activate challenge_mlops
pip install mlflow dvc dvc-s3 pandas scikit-learn fastapi uvicorn
```

Instalamos mlflow, dvc, dvc-s3, pandas, scikit-learn, fastapi y uvicorn en este nuevo entorno.

Git lo configuramos de la siguiente forma:

```
git config --global user.name "LidiaLago"
git config --global user.email "lidialag@ucm.es"
git config --global init.defaultBranch main
```

Creamos una carpeta para el challenge

```
mkdir dir_challenge_mlops && cd dir_challenge_mlops
```

Inicializamos Git y DVC (nos hace falta instalar dvc, porque mi ordenador no reconoce la orden).

```
git init
sudo snap install dvc
dvc init
git commit -m "Iniciando challenge MLOps"
dvc status
```

Si tenemos instalado VS Code y su extensión wsl, podemos ejecutar:

```
code .
```

y abrirlo directamente. Vemos que DVC ha creado los siguientes archivos:

- .dvc (carpeta)
  - tmp (carpeta)
    - btime (archivo)
    - lock (archivo)
  - .gitignore (archivo)
  - config (archivo)
- .dvcignore (archivo)

Estos archivos deben ser agregados y confirmados en Git para establecer la base del seguimiento de datos.

```
git add .dvc
git commit -m "Inicializamos DVC"
```

Ahora vamos a crear el siguiente árbol de carpetas en VS Code por comodidad:

- data/
  - raw/ (no crear) dataset original controlado por DVC
  - processed/ (no crear) datos tras limpieza o ingeniería de características
- src/
- mlruns/
- models/ (almacén de modelos .pkl controlados por dvc)
- notebooks/ (jupyter notebooks para exploración inicial EDA)
- README.md (documentación del proyecto)

Creamos un nuevo fichero .gitignore en la carpeta raíz. Pone lo siguiente:

```
# Entornos virtuales
venv/
.mlops/

# Cachés de Python
pycache/
*.py[cod]

# Datos y Modelos (serán gestionados por DVC)
models/.pkl
models/.joblib

# MLflow local
mlruns/
mlartifacts/

# Secretos
.env
/model.pkl
```

Queremos que git ignore los modelos models/.pkl y models/.joblib porque en MLFlow usaremos
mlflow.sklearn.log_model(model, “model”)
Esto hace que el modelo se gaurde en mlruns/, no hace falta guardarlo en models/
Por eso también tiene sentido ignorar mlruns/ y mlartifacts/

¡Cuidado! Si hago joblib.dump(model, “models/model.pkl”) y también usamos mlflow, tendremos modelos duplicados en models/ y otro en MLflow.

A continuación vamos a configurar nuestro repositorio remoto (que haremos en local):

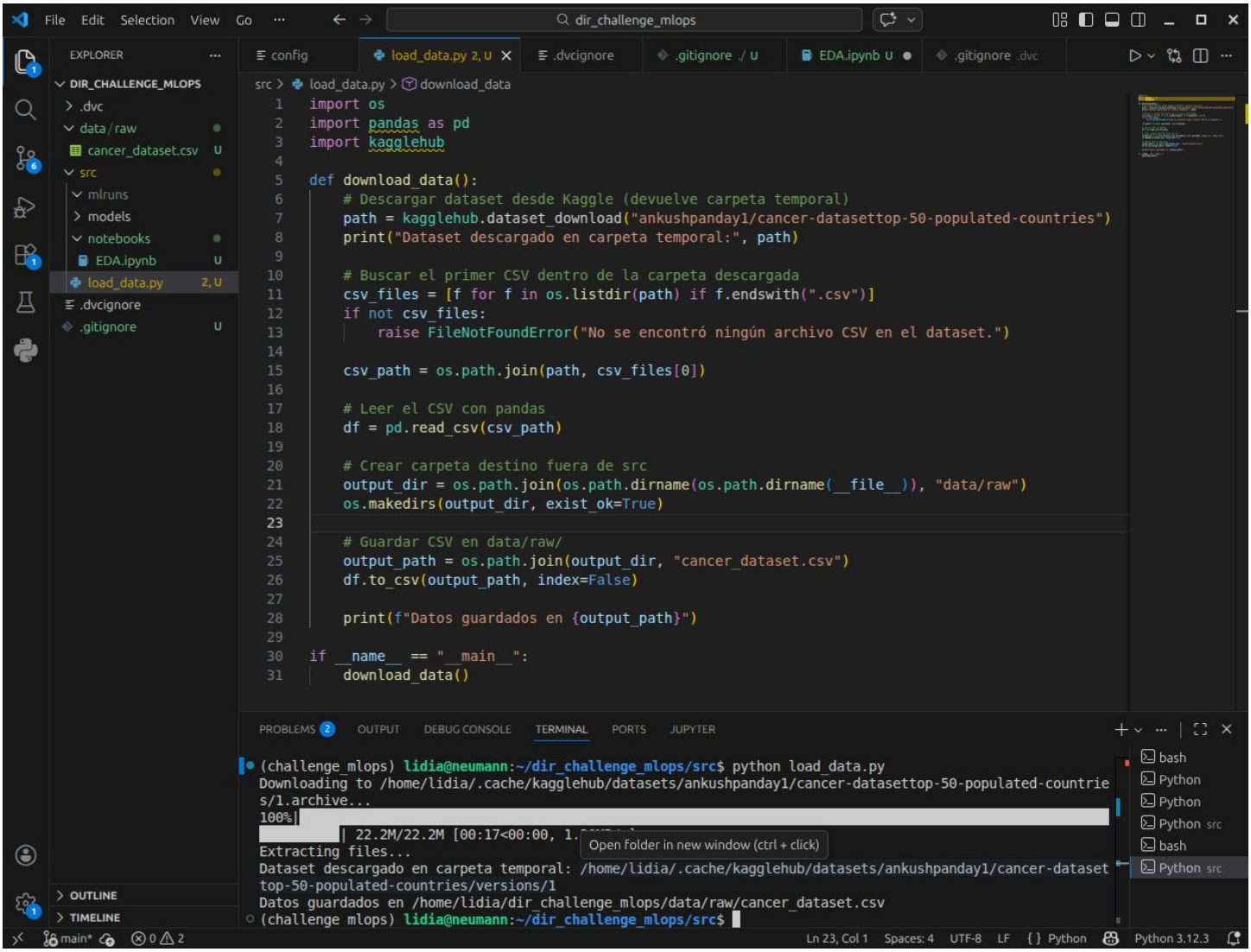
```
mkdir -p /tmp/dvc_remote_storage
dvc remote add -d my_local_remote /tmp/dvc_remote_storage
git add .dvc/config
git commit -m "Configurado storage remoto -local- de DVC"
```

Creamos una carpeta en el sistema tmp/dvc_remote_storage (-p evita errores si ya existe), aquí es donde DVC guardará los datos o modelos pesados (en local). A continuación registramos el almacén en DVC: creamos un remote con nombre 'my_local_remote' en ruta tmp/dvc_remote_storage y lo marcamos como default (-d). Esto se guardará en **.dvc/config** (puedes mirarlo en VS Code). Entonces con git lo versionamos y confirmamos el cambio.

Si vas a VS Code podrás ver en .dvc/config lo siguiente:

```
[core]
    remote = my_local_remote
['remote "my_local_remote"']
    url = /tmp/dvc_remote_storage
```

A continuación vamos a crear un script **src/load_data.py** para cargar los datos. En este challenge usaremos un dataset de Kaggle llamado 'cancer-datasettop-50-populated-countries'. Con este estudiaremos la esperanza de vida de pacientes con cáncer en diferentes países, lo llamaremos cancer_dataset.csv. Su variable objetivo es '**Life_Expectancy**', una variable numérica continua. Tiene 160.000 registros y 23 columnas, entre las cuales podemos destacar:
- Country, Age_Group, Cancer_Type, Risk_Factors, Incidence, Mortality, Urban_population, Obesity_%, Alcohol_consumption_Liters, etc.



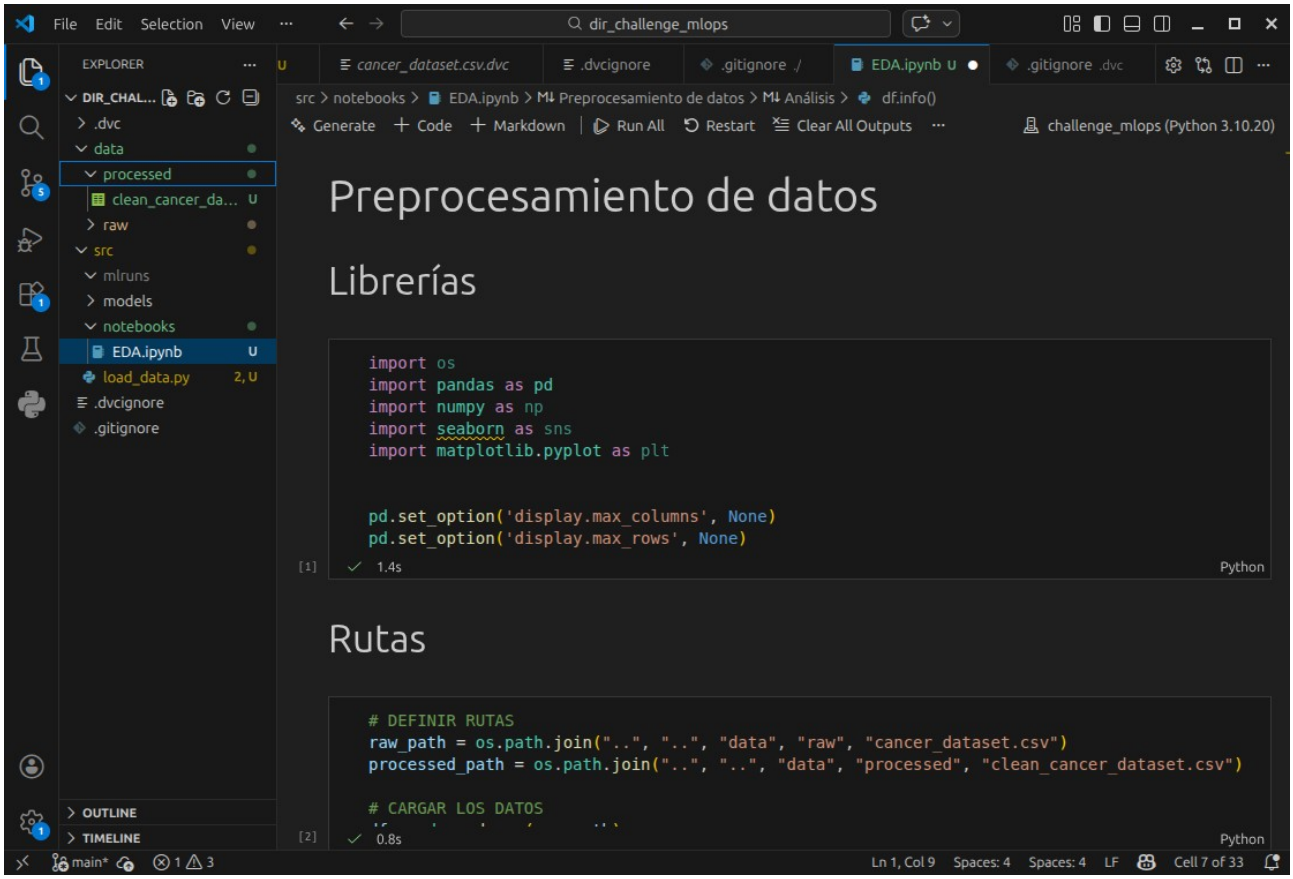
En la imagen anterior podemos ver cómo hemos cargado los datos en formato csv en la carpeta /data/raw/

No podemos olvidar ingestar y versionar los datos:

```
dvc add data/raw/cancer_dataset.csv
git add data/raw/cancer_dataset.csv.dvc .gitignore
git commit -m "Version raw del cancer dataset"
dvc push
```

dvc add csv crea un archivo cancer_dataset.csv.dvc con metadatos de DVC. Git ignorará el csv real, pues lo maneja DVC. Agregamos a Git los metadatos DVC y hacemos commit y push de los cambios. (Puedes comprobar que al ejecutar lo anterior, en la carpeta data se crea un .gitignore con /breast_cancer.csv).

Lo siguiente que vamos a hacer es crear un notebook en /src/notebooks que estudie los datos y los limpie, lo llamamos EDA.ipynb.



Este notebook es únicamente para echarle un primer vistazo a los datos y entender los resultados una vez tengamos el modelo.

Para poder guardar los datos limpios tras el preprocesamiento tenemos que crear una archivo .py, 'preprocess.py' en src. El notebook anterior nos sirve para este propósito, pues habremos estudiado un poco los datos para saber qué preprocesamiento realizar. Una vez limpios, los guardamos en /data/processed y llamamos al archivo clean_cancer_dataset.csv.

src > preprocess.py > remove_outliers

```python
import os
import pandas as pd


def remove_outliers(df, columns=None):
    df_clean = df.copy()

    if columns is None:
        columns = df_clean.select_dtypes(include=["number"]).columns

    for column in columns:
        Q1 = df_clean[column].quantile(0.25)
        Q3 = df_clean[column].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df_clean = df_clean[
            (df_clean[column] >= lower_bound) &
            (df_clean[column] <= upper_bound)
        ]

    return df_clean


def preprocess_data():
    # Ruta
    base_dir = os.path.dirname(os.path.dirname(__file__))

    raw_path = os.path.join(base_dir, "data/raw/cancer_dataset.csv")
    processed_path = os.path.join(base_dir, "data/processed/clean_cancer_dataset.csv")

    # Cargar datos
    df = pd.read_csv(raw_path)
    print(f"Shape inicial: {df.shape}")

    # Limpieza
    df = df.drop_duplicates()
    df = df.dropna()

    # Outliers (solo numéricas)
    df = remove_outliers(df)

    print(f"Shape tras limpieza: {df.shape}")

    # Guardar
    os.makedirs(os.path.dirname(processed_path), exist_ok=True)
    df.to_csv(processed_path, index=False)

    print(f"Dataset limpio guardado en: {processed_path}")


if __name__ == "__main__":
    preprocess_data()
```

Ahora, igual que antes, tenemos que ingestar y versionar los datos.

```
dvc add data/processed/clean_cancer_dataset.csv
git add data/processed/clean_cancer_dataset.csv.dvc .gitignore
git commit -m "Version 1 limpia del cancer dataset"
dvc push
```

A continuación crearemos nuestro archivo python con el modelo en src/train.py. No ponemos captura porque es demasiado largo, pero el archivo se envía adjunto.

El archivo no crea modelo .pkl porque al escribir 'mlflow.sklearn.log_model(pipeline, "model") al final, ya estamos guardando el modelo completo en MLFlow, luego no hace falta guardarlo de nuevo.

Como queremos predecir un número continuo, el modelo que hemos utilizado es un RandomForestRegressor. Tiene las variables 'n_estimators' y 'max_depth' que podremos variar para obtener mejores resultados. Hemos estandarizado variables numéricas, encodeado las categóricas y dividido en conjunto train y test.

Dentro de la función, registramos los parámetros, las métricas y los artefactos (el modelo) con log_param.

En la carpeta raíz (fuera de src) creamos el dvc.yaml, nuestro pipeline de ejecución del modelo. Tiene la siguiente pinta:

```yaml
! dvc.yaml

stages:
  load_data:
    cmd: python src/load_data.py
    deps:
      - src/load_data.py
    outs:
      - data/raw/cancer_dataset.csv

  preprocess:
    cmd: python src/preprocess.py
    deps:
      - src/preprocess.py
      - data/raw/cancer_dataset.csv
    outs:
      - data/processed/clean_cancer_dataset.csv

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed/clean_cancer_dataset.csv

# para ejecutar todo el pipline: dvc repro
```

Para ejecutar el pipeline completo basta con escribir **dvc repro**.

Tienes que eliminar primero los .csv.dvc

```
(challenge_mlops) lidia@neumann:~/dir_challenge_mlops$ dvc repro
Running stage 'load_data':
> python src/load_data.py
Dataset descargado en carpeta temporal: /home/lidia/.cache/kagglehub/datasets/ankushpanday1/cancer-datasettop-50-populated-countries/versions/1
Datos guardados en /home/lidia/dir_challenge_mlops/data/raw/cancer_dataset.csv
Generating lock file 'dvc.lock'
Updating lock file 'dvc.lock'

Running stage 'preprocess':
> python src/preprocess.py
Shape inicial: (160000, 23)
Shape tras limpieza: (160000, 23)
Dataset limpio guardado en: /home/lidia/dir_challenge_mlops/data/processed/clean_cancer_dataset.csv
Updating lock file 'dvc.lock'

Running stage 'train':
> python src/train.py
2026/04/08 18:33:25 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/04/08 18:33:25 INFO mlflow.store.db.utils: Updating database tables
2026/04/08 18:33:27 INFO mlflow.tracking.fluent: Experiment with name 'Life_Expectancy_Optimization' does not exist. Creating a new experiment.
Iniciando experimento: n_estimators=50, max_depth=5
2026/04/08 18:33:43 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/04/08 18:33:43 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's obj
ect serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://
scikit-learn.org/stable/model_persistence.html
RMSE: 10.138124620946941 | R2: -0.0005981416477314205
Iniciando experimento: n_estimators=100, max_depth=10
2026/04/08 18:34:56 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/04/08 18:34:56 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's obj
ect serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://
scikit-learn.org/stable/model_persistence.html
RMSE: 10.140565110625106 | R2: -0.001079935558515821
Iniciando experimento: n_estimators=200, max_depth=20
2026/04/08 18:48:55 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
2026/04/08 18:48:55 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's obj
ect serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://
scikit-learn.org/stable/model_persistence.html
RMSE: 10.14772816352906 | R2: -0.0024947129595940254
Updating lock file 'dvc.lock'

To track the changes with git, run:

        git add dvc.lock

To enable auto staging, run:

        dvc config core.autostage true
Use `dvc push` to send your updates to remote storage.
(challenge_mlops) lidia@neumann:~/dir_challenge_mlops$
```

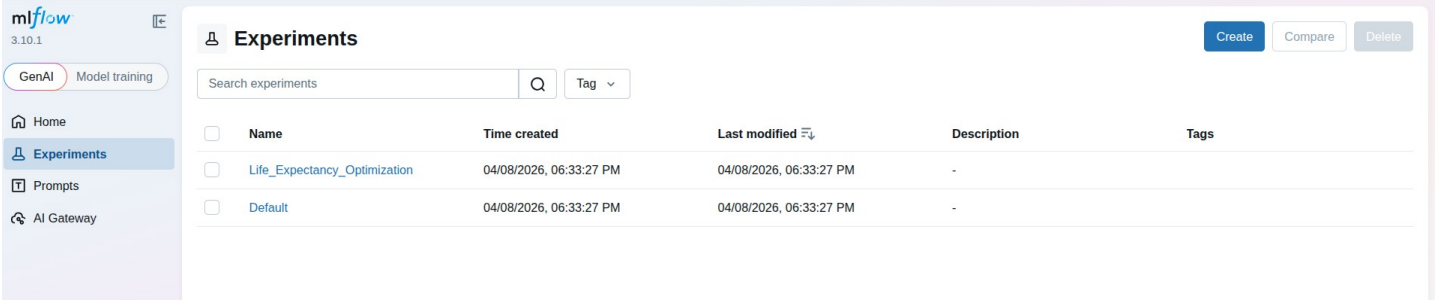
Una vez tenemos el pipeline dvc.yaml, DVC ya no necesita el .dvc individual, los datasets .csv se trackean como output de los stages. Podemos ver que se genera dvc.lock con el hash y control de versiones.

A continuación tenemos que iniciar el sevidor local de MLflow.

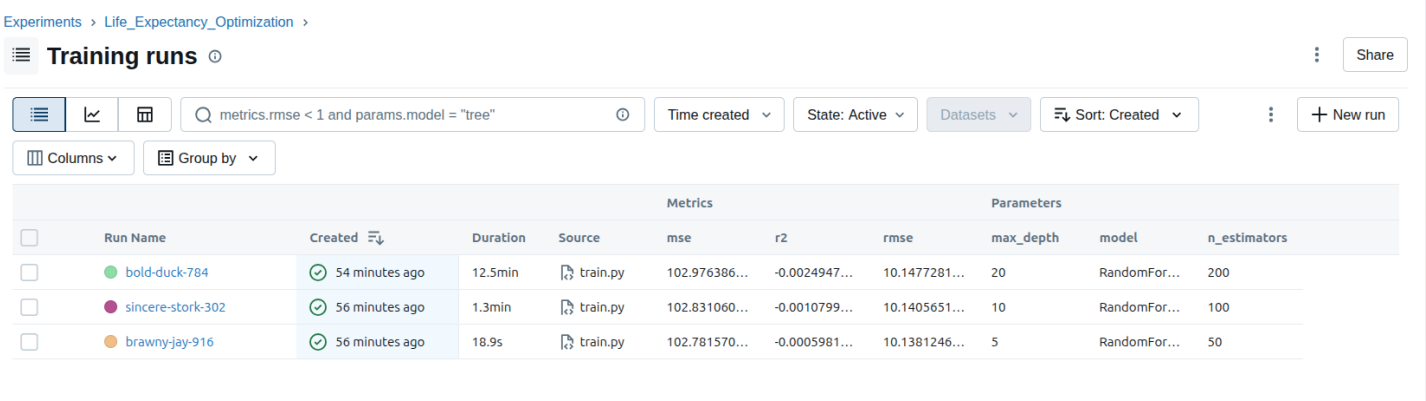
```
conda activate challenge_mlops
mlflow server \
    --backend-store-uri sqlite:///mlflow.db \
    --default-artifact-root ./mlartifacts \
    --host 0.0.0.0 \
    --port 5000
```

Y abrimos el MLflow escribiendo:

```
mlflow ui --port 5000
```



Si nos metemos en Life_Expectancy_Optimization podemos ver todas las ejecuciones de train.py que hemos hecho con distintos parámetros y las métricas obtenidas:



					Metrics			Parameters		
	Run Name	Created	Duration	Source	mse	r2	rmse	max_depth	model	n_estimators
	bold-duck-784	54 minutes ago	12.5min	train.py	102.976386...	-0.0024947...	10.1477281...	20	RandomFor...	200
	sincere-stork-302	56 minutes ago	1.3min	train.py	102.831060...	-0.0010799...	10.1405651...	10	RandomFor...	100
	brawny-jay-916	56 minutes ago	18.9s	train.py	102.781570...	-0.0005981...	10.1381246...	5	RandomFor...	50

El RMSE es la mejor métrica quizás para comparar los runs. (Cuanto más bajo, mejor el modelo). El mejor modelo es el que tiene n_estimator 50 y RMSE 10.13. Sin embargo nos fijamos que R^2 es negativo en todos los casos, es decir, que nuestro modelo no está aprendiendo relaciones útiles. Esto no es nada nuevo, pues como vimos previamente en EDA, las variables no estaban prácticamente relacionadas con la que life_expectancy, y eran poco informativas. Igualmente, vamos a probar con distintos parámetros del modelo para intentar mejorar las métricas.

Vamos a modificar un poco el código train.py para probar con distintos parámetros, por ejemplo, tomamos los siguientes:

```python
# GRID
if __name__ == "__main__":

    param_grid = [
        {
            "n_estimators": 100,
            "max_depth": 5,
            "min_samples_split": 2,
            "min_samples_leaf": 1,
            "max_features": "sqrt"
        },
        {
            "n_estimators": 200,
            "max_depth": 10,
            "min_samples_split": 5,
            "min_samples_leaf": 2,
            "max_features": "sqrt"
        },
        {
            "n_estimators": 300,
            "max_depth": 15,
            "min_samples_split": 10,
            "min_samples_leaf": 4,
            "max_features": "log2"
        },
        {
            "n_estimators": 400,
            "max_depth": None,
            "min_samples_split": 5,
            "min_samples_leaf": 2,
            "max_features": "sqrt"
        }
    ]

    for params in param_grid:
        print(f"Iniciando experimento con: {params}")
        run_experiment(params)
```

Obtenemos los siguientes resultados:

				Metrics			Parameters		
Run Name	Created	Source	Models	mse	r2	rmse	max_depth	max_features	min_sample
lyrical-conch-262	3 hours ago	train.py	random_forest_model	103.072128...	-0.0034267...	10.1524444...	None	sqrt	2
unique-elk-517	3 hours ago	train.py	random_forest_model	102.772422...	-0.0005090...	10.1376734...	15	log2	4
placid-ray-53	3 hours ago	train.py	random_forest_model	102.793304...	-0.0007123...	10.1387033...	10	sqrt	2
upbeat-snipe-540	3 hours ago	train.py	random_forest_model	102.756953...	-0.0003584...	10.1369104...	5	sqrt	1

Como vemos los resultados no varían mucho de los obtenidos en la ejecución anterior. RMSE se mantiene en 10.13 aproximadamente y R^2 es cercano a 0, ligeramente negativo. Esto nos confirma lo que ya sabíamos, que las columnas son irrelevantes para nuestra variable objetivo, son simplemente ruido del que el modelo no puede aprender el comportamiento de Life_Expectancy.

A pesar de esto, despleguemos el modelo. Si tuviésemos que elegir, podríamos quedarnos con

Upbeat-snipe-540:
- RMSE: 10.13691042
- R^2: -0.0003584

- Experiment ID: 330020775659774128
- Run ID: 8d39d1b914f1485f996057bf2d7ebd2c

Para desplegarlo vamos a crear el siguiente archivo dentro de app/main.py:

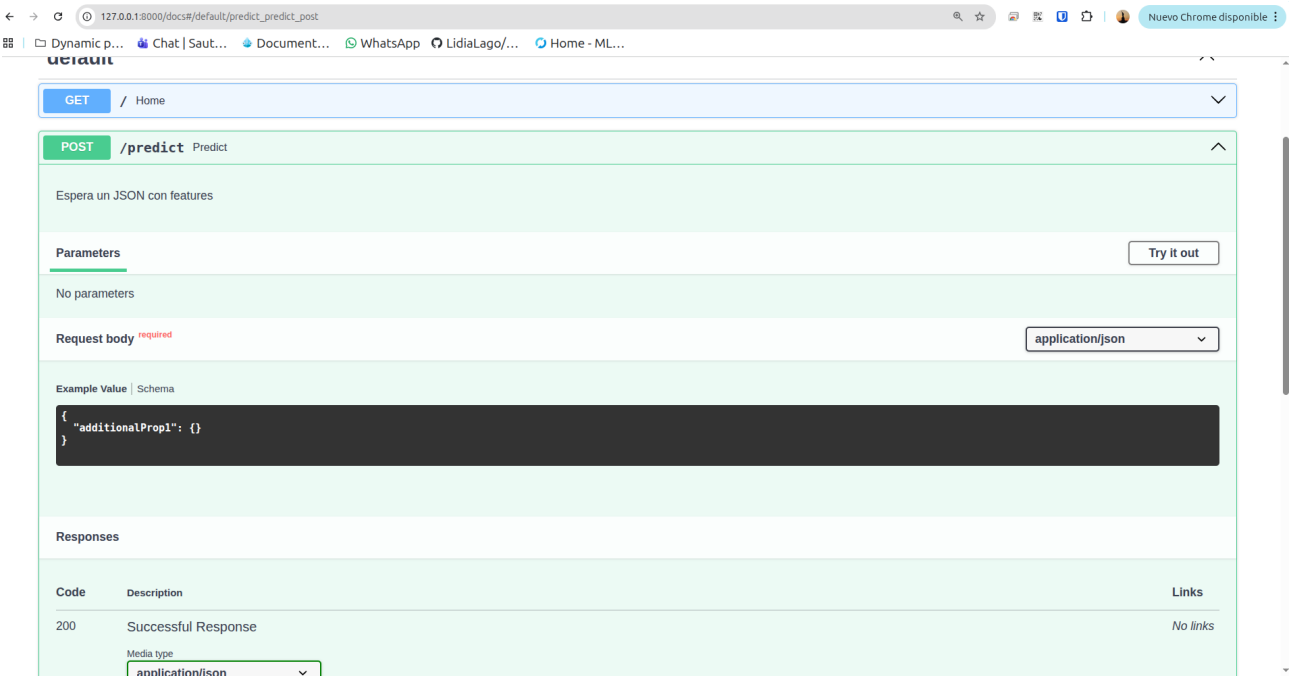
```
app > main.py > ...
from fastapi import FastAPI
import mlflow.pyfunc
import pandas as pd

app = FastAPI()

# Cargar modelo
mlflow.set_tracking_uri("file:./mlruns") # para que fastAPI pueda encontrar el run
RUN_ID = '8d39d1b914f1485f996057bf2d7ebd2c'
MODEL_URI = f"runs:/{RUN_ID}/random_forest_model"
model = mlflow.pyfunc.load_model(MODEL_URI)


@app.get("/")
def home():
    return {"message": "API funcionando"}


@app.post("/predict")
def predict(data: dict):
    """
    Espera un JSON con features
    """
    df = pd.DataFrame([data])
    prediction = model.predict(df)

    return {
        "prediction": prediction.tolist()
    }
```

en model = mlflow.pyfunc.load_model(MODEL_URI) es donde cargamos nuestro modelo entrenado. Luego creamos dos endpoints, el raíz /, que solo tiene un mensaje “API funcionando” y el de predicción /predict donde devolveremos la predicción del modelo al introducirle datos de tipo JSON.

Para ejecutarlo escribimos lo siguiente:

```
uvicorn app.main:app --reload
```

Para ver tu modelo puedes entrar en **http://127.0.0.1:8000/docs**



Puedes probarlo o bien en esa misma página o bien con un curl en la consola. Tomamos unos datos cualesquiera para hacer la predicción:

```
curl -X POST http://127.0.0.1:8000/predict \
-H "Content-Type: application/json" \
-d '{
  "Country":"Turkey",
  "Age_Group":"15-24",
  "Cancer_Type":"Lung",
  "Risk_Factors":"Obesity",
  "Incidence":44,
  "Mortality":457,
  "Prevalence":955,
  "Urban_Population":32.90675830042905,
  "Health_Expenditure_%GDP":11.83400529544704,
  "Tobacco_Use_%":12.578421085596268,
  "Alcohol_Consumption_Liters":1.4459093633093378,
  "Physical_Activity_%":33.19719664120764,
  "Obesity_%":33.944195813190674,
  "Air_Quality_Index":96,
  "UV_Radiation":2.877395416241618,
  "Family_History_%":44.30086194602105,
  "Genetic_Mutation_%":6.924822037196543,
  "Treatment_Coverage_%":97.21091218546712,
  "GDP_per_Capita":29779,
  "Health_Infrastructure_Index":2.3241390992745297,
  "Education_Index":0.8754517690807012,
}'"Population_Density":736.6090059054068
```

Esto devuelve una predicción con el modelo que hemos elegido:

```
(challenge_mlops) lidia@neumann:~$ curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d '{
  "Country":"Turkey",
  "Age_Group":"15-24",
  "Cancer_Type":"Lung",
  "Risk_Factors":"Obesity",
  "Incidence":44,
  "Mortality":457,
  "Prevalence":955,
  "Urban_Population":32.90675830042905,
  "Health_Expenditure_%GDP":11.83400529544704,
  "Tobacco_Use_%":12.578421085596268,
  "Alcohol_Consumption_Liters":1.4459093633093378,
  "Physical_Activity_%":33.19719664120764,
  "Obesity_%":33.944195813190674,
  "Air_Quality_Index":96,
  "UV_Radiation":2.877395416241618,
  "Family_History_%":44.30086194602105,
  "Genetic_Mutation_%":6.924822037196543,
  "Treatment_Coverage_%":97.21091218546712,
  "GDP_per_Capita":29779,
  "Health_Infrastructure_Index":2.3241390992745297,
  "Education_Index":0.8754517690807012,
  "Population_Density":736.6090059054068
}'
{"prediction":[67.36826897984962]}(challenge_mlops) lidia@neumann:~$
```